

# Branch predictors and speculative execution

## Homework 4

### Part 1: Hybrid Branch Predictor

I added a structure to keep track of the 32 last predictions of each predictor and choose the one that had the best score in the past.

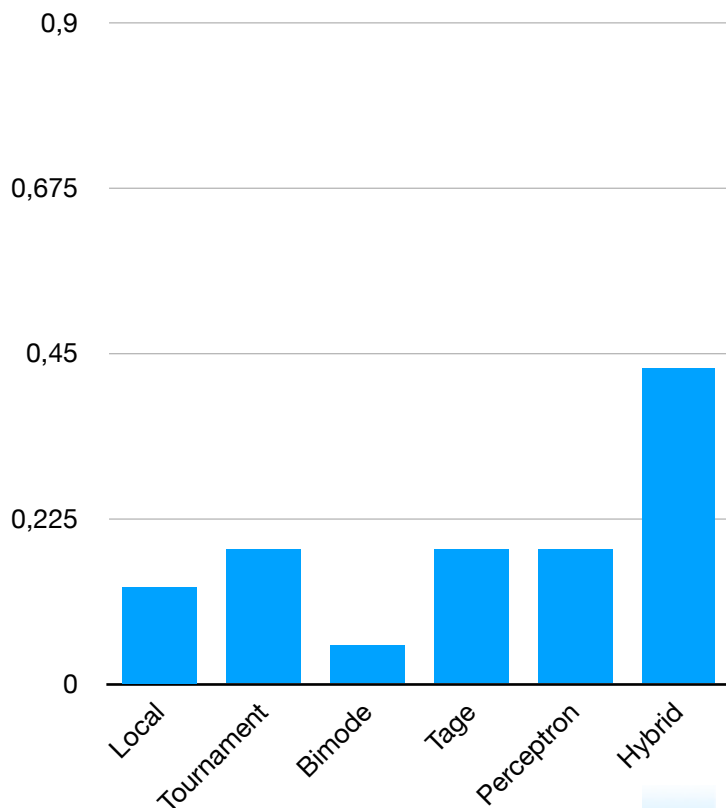
```
struct AccuracyTracker {
    int history_window_length = 32;
    std::deque<bool> localCorrectHist;
    std::deque<bool> tournamentCorrectHist;
    std::deque<bool> bimodeCorrectHist;
    std::deque<bool> tageCorrectHist;
    std::deque<bool> perceptronCorrectHist;

    void updateAccuracyHistory(std::deque<bool>& history, bool
wasCorrect) {
        if (history.size() == history_window_length) {
            history.pop_front();
        }
        history.push_back(wasCorrect);
    }

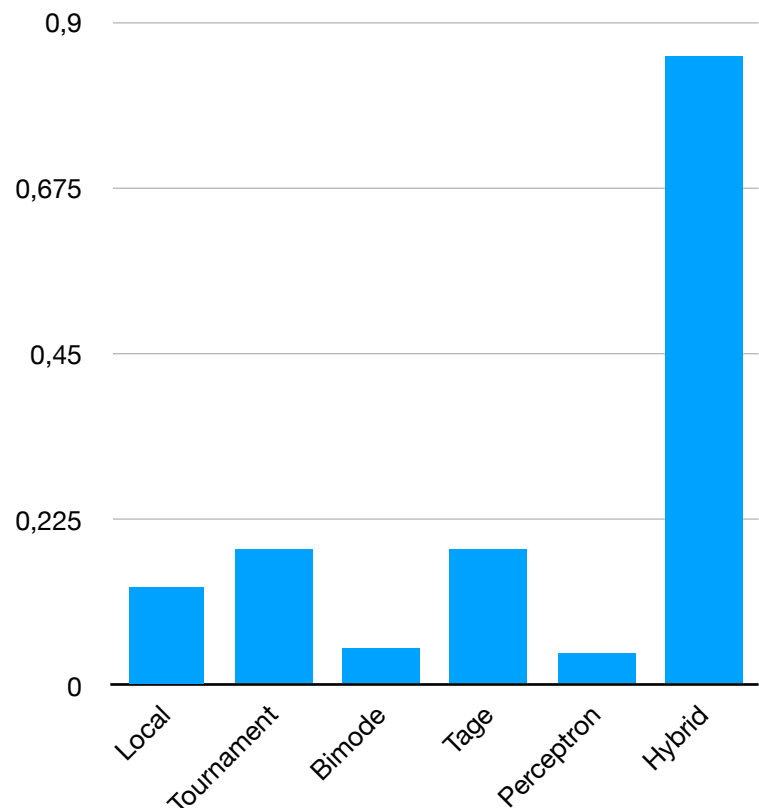
    int getScore(const std::deque<bool>& history) const {
        return std::count(history.begin(), history.end(), true);
    }
};
```

## Part 2: Analysis of the branch predictors

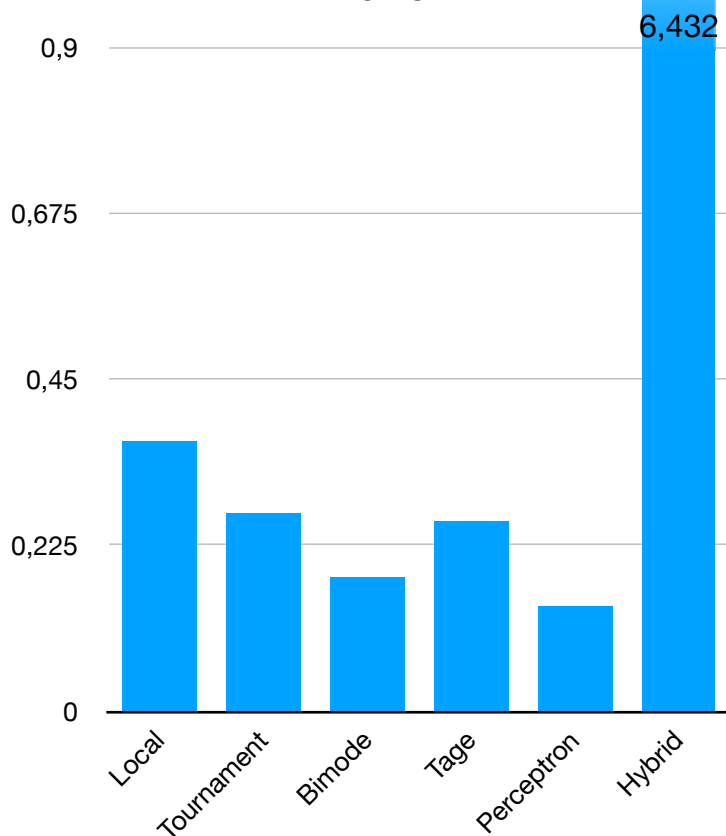
### Incorrect Prediction rate on CCa



### Incorrect Prediction rate on CCm



### Incorrect Prediction rate on CRm



The standalone predictors have a lower misprediction rate than the hybrid predictor. However, this discrepancy is variable, and is highly dependent on the task in hand.

For the **CCa** program, the **bimode** predictor is the best, outperforming the hybrid by a factor of 8.4.

For the **CCm** program, the **perceptron** predictor is the best, outperforming the hybrid by a factor of 20.3.

For the **CRm** program, the **perceptron** predictor is the best, outperforming the hybrid by a factor of 45.3.

## Part 2: Analysis of the branch predictors

**A:** While each standalone predictor has its own way to keep history (very short-term and per-PC for local, dual paths for tournament, separate taken/not-taken biasing in bimode, long and multi-length global history in TAGE, and perceptron-based correlation with global history), the hybrid causes a uniformization of their behavior, especially in terms of how predictions are scheduled. Because all predictors are invoked for every branch in the hybrid, their internal histories may evolve less optimally. This disrupts their normal specialization and training patterns, and reduces the performance.

**B:** Within the constraints of area, cost, or energy consumption, designers must allocate resources to the components that offer the greatest performance gains for a given objective. In an existing out-of-order speculative superscalar CPU, unless we're targeting a branch-heavy application, it may be more beneficial to optimize other parts of the processor.

- For example, we could dedicate more silicon area to data dependency prediction, enabling the processor to speculatively execute consecutive instructions in parallel by predicting that they are independent. A dedicated control unit could monitor execution and trigger a refetch if a misprediction is detected, while correct predictions would yield speedups.
- Another direction would be enhancing the instruction scheduling logic. With additional transistors, we could improve the CPU's ability to dynamically reorder and cluster instructions, ensuring that functional units remain busy regardless of instruction type. This would reduce idle time and improve overall throughput by better utilizing the processor's parallel resources.

**C:** TAGE is designed for environments with diverse real-world applications with deeply nested control flow, indirect branches, or input-dependent patterns. These scenarios allow TAGE's multiple predictor tables with varying history lengths to collectively capture correlations across both short and long temporal distances.

However, the CCa, CCm, and CRm benchmarks used in this evaluation do not offer this kind of branching behavior:

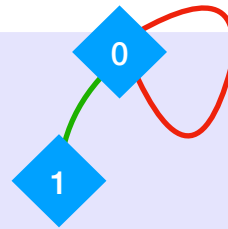
- CCa (Completely Biased Branches) contains branches that are entirely predictable. This means that simple predictors like bimodal can learn the pattern after just one or two executions. TAGE is unnecessarily complicated, and may even hurt performance due to longer prediction latency or initialization time.
- CCm (Heavily Biased Branches) includes branches that are *mostly* taken or not taken, with occasional exceptions. While slightly more complex than CCa, this benchmark still falls within the domain where short-history or biased predictors excel. TAGE's long history tracking and tag comparisons don't significantly help here, since the dominant pattern is easy to learn and maintain with simpler structures.
- CRm (Merge Sort) is more dynamic and involves data-dependent control flow due to the sorting algorithm, but the branching behavior tends to be regular and tied to recursive structure or loop bounds. The correlations are medium range. As a result, TAGE's deeper history tables again provide limited benefit over lighter predictors that can pick up on loop behavior quickly.

These benchmarks do not present the unstructured, data-correlated, or deeply history-dependent branches that make TAGE truly stand out in real life.

## Part 3: Alternatively speculative execution

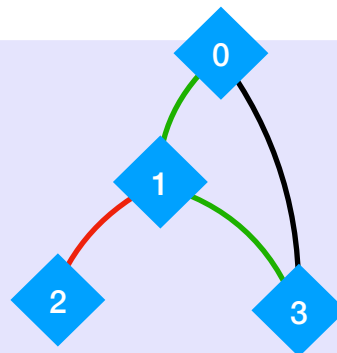
A:

```
0:  goto line1 //pred: goto line0
1:  end
```



In this code, if the jump predicts line0, the SemPE protocol could risk the creation of an infinite growing tree with an exponential number of branches.

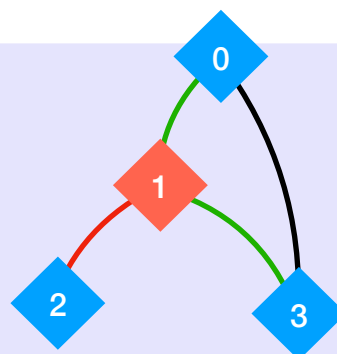
```
0:  goto line3 //pred: goto line1
1:  goto line1 //pred: goto line3
2:  incorrect end
3:  end
```



In this situation, if the prediction at line 0 is line1, followed by a prediction at line 1 which is line3, the result (green) is equivalent as one direct jump to line3 (black).

To resolve the misprediction, we would only need to kill the red hart, without needing to spawn a new one.

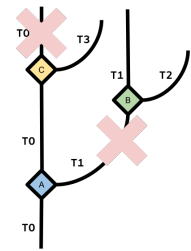
```
0:  goto line3 //pred: goto line1
1:  res_value++
   goto line1 //pred: goto line3
2:  incorrect end
3:  end
```



This situation is the similar to the previous1, but there, the alternative green path (instead of black) has an impact on the result of the execution, with the value of res\_value being incremented incorrectly.

To resolve the misprediction, we would only need to kill the red and green harts, and would need to spawn a new one, corresponding to black.

**B:** It is possible to have the branch C evaluated before the branch B is reached in an O3 processor if T1 is composed of very long instructions to compute (multiple memory accesses for example) while T0 is short. In that case, we need a control logic that will keep track of potential modifications in memory in T3 that could be used in T1 and would cause an incoherence if not reverted.



**C:** In a O3 implementation of P3, on a branch, we have to track what branch is predicted and which one is taken. We then need to keep track of the data that is modified and that can potentially be appear modified, while it should not. In that case, we should squash the not taken ranch and revert any wrong modification of data. We would have to traceback until the first incorrectly predicted branch. Structural hazards of branches could be branches that call themselves and create an infinite loop.

**D:** SeMPE will execute both sides of the branch, so it is more computationally intensive than the hart-spawning processor, that in case of good predictions spares the computation of the not taken branch. On the flip side, SeMPE doesn't give any information about the side of the branched actually used by the program, while the hart-spawning processor can leak information about the computation being executed, because the timing can vary between the two branches.

**E:** I agree that SeMPE can help address security needs, especially on shared systems like those in the cloud. However, it introduces additional computational overhead, so it's not always necessary. I believe a better approach would be to protect only the critical parts of the code. Also, implementing it in hardware reduces flexibility for users, so a software-based implementation might be more suitable for a general-purpose computer.

For the example of the ISA assignment, we could implement SeMPE in software easily, without needing highly specific hardware:

```
if (strcmp(argv[1], "supersecretdata") == 0) {
    for (int i = 0; i < 65536; i++) {
        count += r1 * r2;
    }
} else if (strcmp(argv[1], "notsosecretdata") == 0) {
    for (int i = 0; i < 65536; i++) {
        count += r1 / r2;
    }
}
```

**ORIGINAL**

```
for (int i = 0; i < 65536; i++) {
    count1 += r1 * r2;
    count2 += r1 / r2;
}

if (strcmp(argv[1], "supersecretdata") == 0) count = count1;
else if (strcmp(argv[1], "notsosecretdata") == 0) count = count2;
```

**PROTECTED AGAINST TIMING  
SIDE-CHANNELS WITH SEMPE**